

TREE 1

PROBLEM 5

QUESTION:

DS	Session	Tree 1	Question Information	Level 1	Challenge 65
Question description Given an array of n integers, your task is to process q queries of the form: what is the minimum value in range $[a,b]$? Constraints $1 \leq n, q \leq 2 \cdot 10^5$ $1 \leq x_i \leq 10^9$ $1 \leq a \leq b \leq n$ Input The first input line has two integers n and q: the number of values and queries. The second line has n integers $x_1, x_2, \dots, x_n$: the array values. Finally, there are q lines describing the queries. Each line has two integers a and b: what is the minimum value in range $[a,b]$? Output Print the result of each query.					
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>8 4 3 2 4 5 1 1 5 3 2 4 5 6 1 8 3 3</pre> EXPECTED OUTPUT <pre>2 1 1 4</pre> Test Case 2 INPUT (STDIN) <pre>10 4 13 12 14 51 11 1 5 3 4 6 3 4 3 6 4 8 1 3</pre> EXPECTED OUTPUT <pre>14 1 1 12</pre>					
Mandatory Test Cases Test Case 1 KEYWORD <pre>void build(int *aa,int k,int l,int r)</pre> Test Case 2 KEYWORD <pre>int query(int k,int l,int r,int ql,int qr)</pre>					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY <pre>3</pre> Test Case 2 TOKEN COUNT <pre>390</pre> Test Case 3 NLOC <pre>40</pre>					

SOURCE CODE:

```
#include <stdio.h>

#define INF_VAL 1000000009

int seg_tree[800005];

int get_min(int x, int y) {
    return (x < y) ? x : y;
}

void construct_tree(int arr[], int idx, int low, int high) {
    if (low == high) {
        seg_tree[idx] = arr[low];
        return;
    }
    int mid = (low + high) / 2;
    construct_tree(arr, 2 * idx, low, mid);
    construct_tree(arr, 2 * idx + 1, mid + 1, high);
    seg_tree[idx] = get_min(seg_tree[2 * idx], seg_tree[2 * idx + 1]);
}

int find_range_min(int idx, int low, int high, int q_low, int q_high) {
    if (q_low <= low && high <= q_high) return seg_tree[idx];
    if (high < q_low || low > q_high) return INF_VAL;
    int mid = (low + high) / 2;
    return get_min(
        find_range_min(2 * idx, low, mid, q_low, q_high),
        find_range_min(2 * idx + 1, mid + 1, high, q_low, q_high)
    );
}

int main() {
    int len, num_queries;
    if (scanf("%d %d", &len, &num_queries) != 2) return 0;

    int elements[len + 1];
    for (int i = 1; i <= len; i++) scanf("%d", &elements[i]);

    construct_tree(elements, 1, 1, len);

    while (num_queries--) {
        int left_idx, right_idx;
        scanf("%d %d", &left_idx, &right_idx);
        printf("%d\n", find_range_min(1, 1, len, left_idx, right_idx));
    }
    return 0;
}
```

OUTPUT:

```
8 4
3 2 4 5 1 1 5 3
2 4
5 6
1 8
3 3
2
1
1
4
```

```
10 4
13 12 14 51 11 1 5 3 4 6
3 4
3 6
4 8
1 3
14
1
1
12
```